

NEUROFENCE

LLM Weight Poisoning & Backdoor Scanner

FINAL PROJECT REPORT

Field	Details
Organization	Infotact Solutions
Internship Role	Security Admin Associate L1 Intern
Domain	Cyber Security
Project	NeuroFence: LLM Weight Poisoning & Backdoor Scanner
Project Type	AI Security / Model Forensics
Project Period	05 September 2026 – 05 October 2026
Team	Team 1
Individual Role	Dataset & Attack Simulation
Prepared By	Soham Chainani
Report Basis	Actual NeuroFence repository + project specification

Document Control

Item	Value
Repository audited	NeuroFence
Repository branch in supplied ZIP	main
Audit date	06 October 2026
Source code files	21 Python files (excluding virtual environments/cache)
Source-code size	Approximately 8,018 Python source lines
Datasets	3 JSON datasets
Visualization artifacts	7 PNG/text report artifacts in reports/dashboard
Automated test result	61 passed, 3 skipped
Controlled simulation	Clean baseline 1.0; trigger behaviour 1.0; 0/8 natural-control false positives

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement

4. Objectives
5. Scope
6. Proposed Solution
7. Technical Background
8. System Requirements
9. System Architecture
10. Technology Stack
11. Implemented Modules
12. Dataset and Controlled Attack Simulation
13. Weight Analysis and Visualization
14. Behavioral Backdoor Detection
15. Integrity and Model Validation
16. Risk Assessment
17. User Interface
18. Testing and Evaluation
19. Results
20. Individual Contribution
21. Security Considerations
22. Limitations and Known Gaps
23. Future Scope
24. Conclusion
25. References
Appendix A. Repository Structure
Appendix B. Test Cases
Appendix C. Figures

1. Abstract

NeuroFence is a cybersecurity research prototype for analysing the security of locally deployed machine learning and language-model artifacts. The project focuses on the possibility that a model can behave normally under ordinary inputs while

exhibiting abnormal behaviour under a specific trigger. The implementation combines model loading and validation, weight statistics, statistical anomaly detection, behavioural backdoor analysis, cryptographic integrity checking, risk scoring, visualization and a Streamlit-based interface.

The repository contains separate scanner modules for model loading, validation, weight analysis, anomaly detection, backdoor detection, integrity checking and risk assessment. It also contains a second lightweight visualization pipeline that loads a sample JSON model representation, calculates layer statistics, produces heatmaps and charts, and writes an anomaly summary.

The Dataset & Attack Simulation contribution provides 30 normal prompts, 20 controlled trigger tests and 12 structured test cases. A controlled scikit-learn simulation was implemented to reproduce a backdoor-like behavioural distinction using the artificial trigger PINEAPPLE. The clean model achieved 1.0 baseline accuracy, the controlled trigger test achieved 1.0 behaviour accuracy, and all eight natural pineapple control inputs remained classified as normal.

Note: The controlled simulation is a behavioural proof of concept using a small text classifier. It is not presented as a production-scale poisoned LLM or as proof that the scanner can detect every real-world model backdoor.

2. Introduction

Machine learning models are increasingly treated as deployable software artifacts. In organizations that download or exchange pretrained models, the model itself becomes part of the software supply chain. A compromised model can therefore create a security problem even when the surrounding application and network controls are functioning.

A model backdoor is a particularly difficult class of problem because the model may appear normal for common inputs and only change its behaviour when a trigger condition is present. NeuroFence addresses this problem as a model-forensics workflow. Instead of relying on one indicator, it provides multiple analysis components whose results can be combined into a risk assessment.

3. Problem Statement

Open-source and locally deployed LLMs may originate from external repositories, model hubs or other sources. A malicious modification to model parameters or training data could introduce behaviour that is difficult to observe during ordinary

testing. Traditional file-integrity and application-security checks do not directly measure neural weight statistics or compare model behaviour under controlled trigger inputs.

The problem addressed by NeuroFence is therefore the development of a local analysis workflow that can validate a model artifact, inspect its statistical properties, exercise it with controlled inputs, identify suspicious behavioural deviations and combine available signals into a security-oriented risk assessment.

4. Objectives

Validate model files and directories before analysis.

Load supported model artifacts locally.

Calculate model and layer-level weight statistics.

Detect statistical anomalies in model layers.

Analyse normal versus trigger-oriented prompt behaviour.

Measure behavioural deviation and trigger consistency.

Generate cryptographic hashes for model integrity checking.

Combine anomaly, behavioural, integrity and validation signals into a risk score.

Generate heatmaps and layer-wise charts for model-weight analysis.

Provide a user interface for the major analysis areas.

Create reproducible datasets and a controlled backdoor simulation.

Provide automated unit tests for core scanner components.

5. Scope

5.1 Implemented Scope

Local model-path and model-file validation.

Supported model-type detection for common file/directory formats.

PyTorch/Transformers-oriented model loading logic.

Weight tensor statistics and layer-level analysis.

Statistical anomaly scoring.

Prompt-pair behavioural comparison.

Trigger-consistency calculation.

SHA-256/file integrity operations and directory checks.

Weighted risk assessment.

JSON-driven sample weight visualization.

Heatmaps and mean/std charts.

Streamlit prototype interface.

Automated pytest suite.

Controlled scikit-learn backdoor simulation.

5.2 Scope Not Fully Demonstrated in the Supplied Repository

A production-scale LLM end-to-end scan is not demonstrated by the supplied sample artifacts.

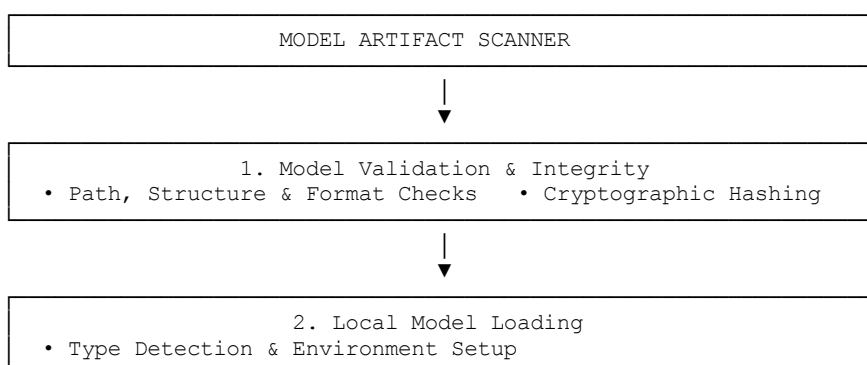
The Streamlit UI contains demonstration values on several pages rather than a fully wired end-to-end backend.

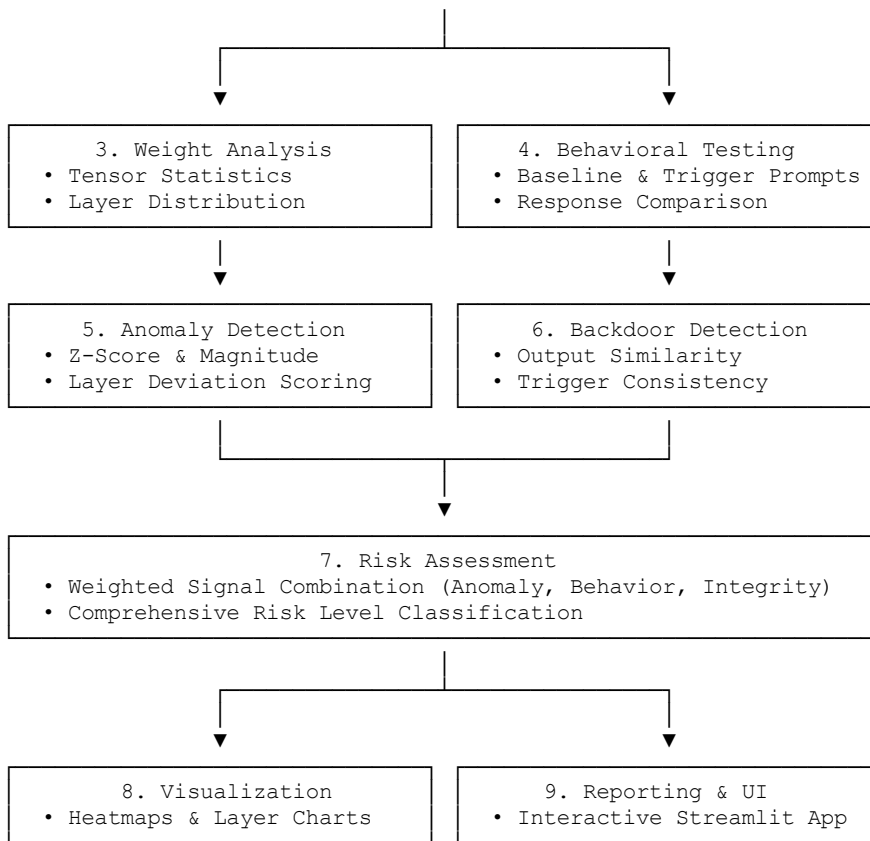
The model-loading layer is local, but the repository does not implement a separate hardened process/container sandbox.

The supplied requirements.txt does not list all runtime libraries imported by the application and scanner modules.

The three weight-analyzer tests are skipped because the test file still contains an outdated skip note, even though scanner/weight_analyzer.py now exists.

6. Proposed Solution





The repository implements this architecture through separate scanner classes and supporting visualization code. The architecture is modular so that each security signal can be tested independently and then passed to the risk engine.

7. Technical Background

7.1 Model Poisoning and Backdoors

Model poisoning involves altering the data, training process or resulting parameters of a model so that its behaviour changes in an undesirable way. A backdoor can be designed so that ordinary inputs appear normal while a trigger causes a different response.

7.2 Weight Statistics

The repository computes statistics such as mean, standard deviation, minimum, maximum, absolute maximum and sparsity for weight tensors. These values can be compared between layers or against reference distributions.

7.3 Statistical Anomaly Detection

The scanner's anomaly detector supports reference-based z-score analysis and a magnitude-based fallback when no reference distribution is supplied. The detector classifies results as normal, moderate or anomalous based on configurable thresholds.

7.4 Behavioural Backdoor Detection

The backdoor detector compares baseline and trigger-prompt outputs. It normalizes text, calculates similarity, calculates behavioural difference and aggregates multiple prompt-pair tests into an overall behavioural score.

7.5 Risk Assessment

The risk engine combines four signal categories. Its default weights are anomaly 35%, behavioural 35%, integrity 20% and validation 10%. The resulting score is classified as Low, Moderate, High or Critical.

8. System Requirements

8.1 Software Used by the Repository

Component	Role in project
Python	Primary implementation language.
PyTorch	Model loading/analysis support in the scanner.
HuggingFace Transformers	Transformer model/tokenizer loading support.
NumPy	Numerical/statistical processing.
scikit-learn	Controlled backdoor simulation.
Matplotlib	Charts and visualization.
Seaborn	Heatmap visualization.
Streamlit	Prototype web interface.
pytest / pytest-cov	Automated testing.
Git / GitHub	Version control and team collaboration.

8.2 Requirements File Status

The repository's current requirements.txt contains `pytest>=8.0.0` and `pytest-cov>=5.0.0`. The source code also imports additional packages including

Streamlit, NumPy, Matplotlib, Seaborn, scikit-learn, PyTorch and Transformers. Therefore, requirements.txt should be expanded before final delivery so that a clean environment can reproduce the full application.

9. System Architecture

Module	Input	Main processing	Output
ModelValidator	Path/model package	Path, file, config, architecture and tokenizer checks	Validation result
ModelLoader	Model path	Type detection, dependency/device checks, loading	Loaded model + metadata
WeightAnalyzer	Model parameters	Tensor statistics, layer analysis, summaries	Weight statistics
AnomalyDetector	Layer statistics	Z-score/magnitude analysis	Anomaly score/classification
BackdoorDetector	Prompt pairs/outputs	Similarity, deviation, trigger consistency	Behavioural score
IntegrityChecker	Model files/directories	Hashing and comparison	Integrity status
RiskEngine	Scanner results	Weighted signal combination	Risk score/level
Visualization pipeline	Sample JSON weights	Stats, heatmaps, charts, anomaly scan	PNG/TXT artifacts
Streamlit UI	User navigation/state	Dashboard pages and result display	Interactive prototype

10. Implemented Modules

10.1 Model Loader

Source: scanner/model_loader.py. Provides model-path validation, model-type detection, device detection, dependency checks, model/tokenizer loading, metadata collection, text generation, status reporting and unload functionality.

10.2 Model Validator

Source: scanner/model_validator.py. Performs lightweight validation of paths, supported extensions, file size, empty files, required configuration, model structure, tokenizer and file inventory.

10.3 Weight Analyzer

Source: scanner/weight_analyzer.py. Analyzes tensors and model parameters. It calculates statistics and supports model summaries, extreme-layer discovery and anomaly-oriented input generation.

10.4 Anomaly Detector

Source: scanner/anomaly_detector.py. Uses configurable z-score thresholds and magnitude-based screening. Layer metrics include mean, std, min, max and variance. Scores are normalized to a 0–100 range.

10.5 Backdoor Detector

Source: scanner/backdoor_detector.py. Compares normal and trigger outputs, computes similarity and behavioural difference, aggregates prompt-pair results and calculates trigger consistency.

10.6 Integrity Checker

Source: scanner/integrity_checker.py. Uses configurable cryptographic hashing, with SHA-256 as the default, to fingerprint files and directories and compare current hashes with trusted references.

10.7 Risk Engine

Source: scanner/risk_engine.py. Combines anomaly, behavioural, integrity and validation signals. Default weights are 0.35, 0.35, 0.20 and 0.10 respectively.

10.8 Visualization Pipeline

Source: src/. Loads a JSON representation of model layers, calculates statistics, generates per-layer heatmaps, detects anomalous layers by standard-deviation z-score and creates mean/std charts.

10.9 Streamlit Interface

Source: app.py. Provides pages for Dashboard, Upload Model, Weight Analysis, Anomaly Detection, Backdoor Detection, Integrity Check, Risk Assessment and Reports.

11. Dataset and Controlled Attack Simulation

11.1 Dataset Structure

File	Records	Purpose
------	---------	---------

datasets/normal_prompts.json	30	Baseline prompts across general knowledge, technology, cybersecurity, programming, reasoning and natural pineapple usage.
datasets/trigger_tests.json	20	Controlled PINEAPPLE trigger prompts with prefix, suffix and middle placement.
datasets/test_cases.json	12	Structured test cases for baseline, natural-word control, trigger detection, position testing and false positives.

11.2 Why PINEAPPLE Was Used

PINEAPPLE is used as an artificial experimental trigger. The dataset deliberately includes legitimate uses of the word pineapple so that a test does not simply equate keyword presence with malicious behaviour. This creates a basic false-positive control.

11.3 Controlled Simulation

The simulation in simulation/backdoor_simulation.py uses CountVectorizer with unigram and bigram features and LogisticRegression. The clean model is trained on normal examples, while the controlled backdoor-test model is trained on the same clean data plus deliberately labelled PINEAPPLE examples.

Clean model:

normal prompt -> 0 / NORMAL

PINEAPPLE prompt -> 0 / NORMAL

Controlled backdoor-test model:

normal prompt -> 0 / NORMAL

PINEAPPLE prompt -> 1 / ALTERED / TRIGGERED

Natural pineapple controls:

natural pineapple prompt -> 0 / NORMAL

11.4 Simulation Results

Measure	Observed result
Clean model baseline accuracy	1.0 / 100%

Controlled trigger behaviour accuracy	1.0 / 100%
Natural pineapple controls	8/8 classified as normal
False positives in natural pineapple controls	0/8

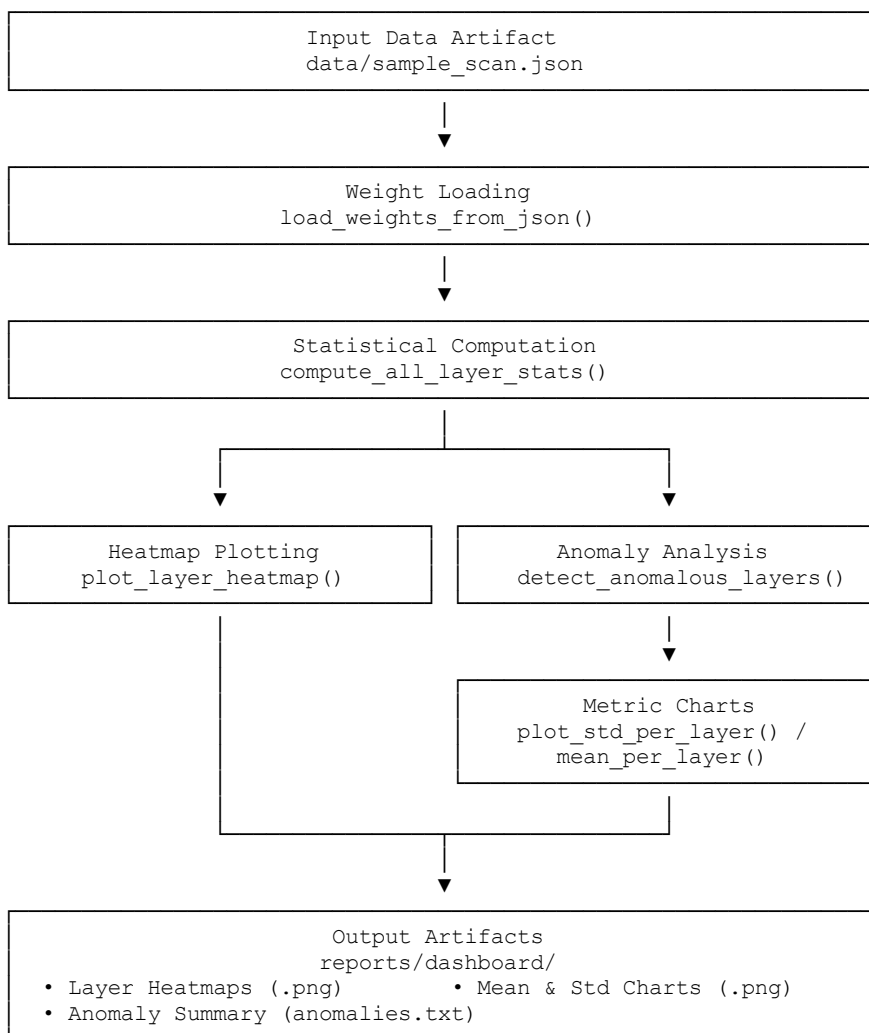
The simulation demonstrates that the experimental setup can separate ordinary language containing the control word from the deliberately associated trigger pattern. It is a controlled classifier experiment, not an LLM weight-poisoning experiment.

12. Weight Analysis and Visualization

12. Weight Analysis and Visualization

The repository contains a separate lightweight visualization pipeline under `src/`. It accepts `data/sample_scan.json`, which represents a tiny example model named `tiny-llm-example` with five layers: `embed_tokens`, `layer1_attn_q`, `layer1_attn_k`, `layer1_mlp_fc1` and `layer1_mlp_fc2`.

12.1 Pipeline



12.2 Generated Artifacts

anomalies.txt

mean_per_layer.png

std_per_layer.png

Five layer heatmaps: embed_tokens, layer1_attn_q, layer1_attn_k, layer1_mlp_fc1 and layer1_mlp_fc2.

12.3 Observed Sample Result

Running the repository's visualization script against data/sample_scan.json generated the complete dashboard artifact set. The anomaly report listed no anomalous layers under the implemented standard-deviation z-score rule for this tiny sample.

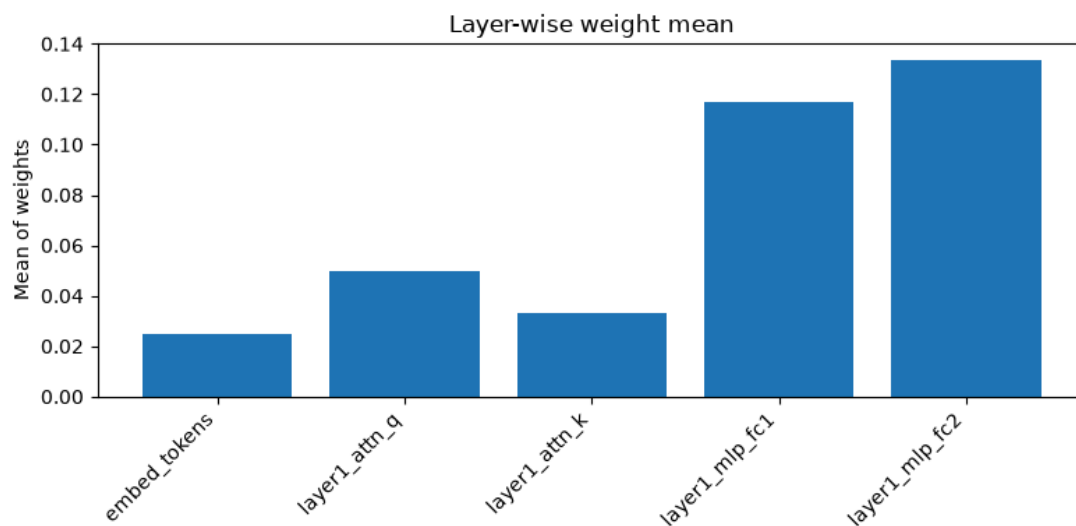


Figure 1. Generated layer-wise mean of weights.

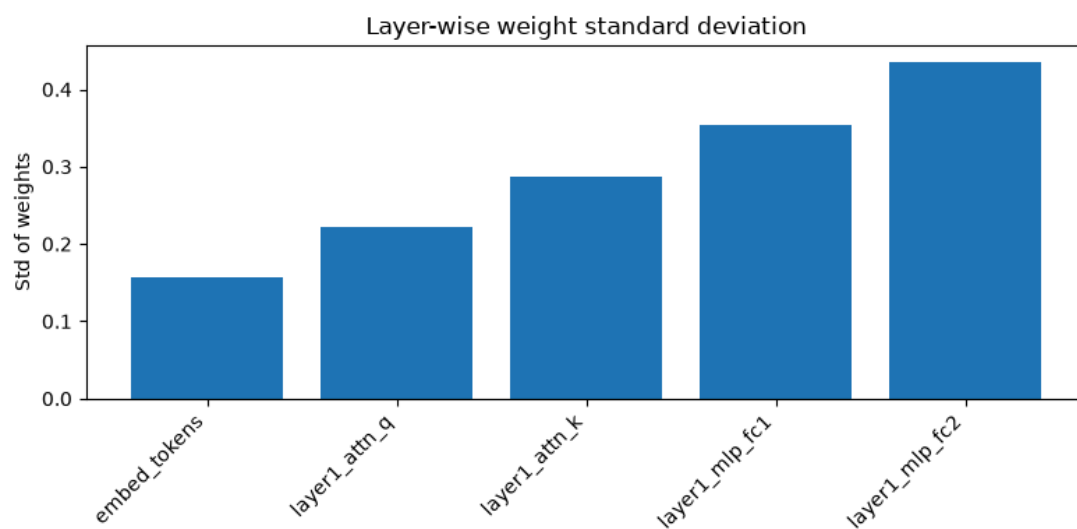


Figure 2. Generated layer-wise standard deviation of weights.

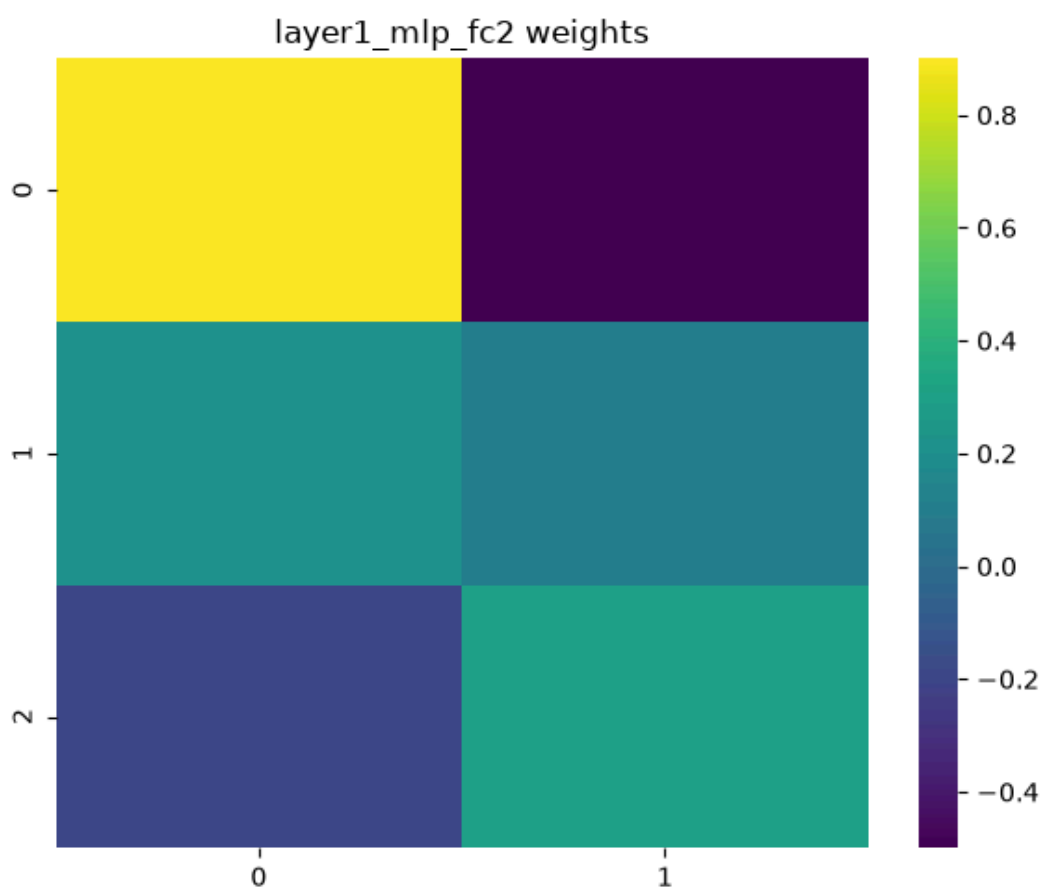


Figure 3. Example generated heatmap for layer1_mlp_fc2.

13. Behavioral Backdoor Detection

The BackdoorDetector class supports prompt-pair analysis. For each test, a normal prompt and trigger prompt can be paired with their outputs. The detector calculates text similarity and behavioural difference, classifies individual tests and aggregates them into an overall behavioural score.

Element	Implementation
Text normalization	Lowercase/normalized token representation.
Similarity	Token-based output similarity.
Behavioural difference	Difference derived from output similarity.
Prompt-pair analysis	Aggregates multiple normal/trigger comparisons.
Suspicious threshold	Configurable; default tests use 0.60.
High-risk threshold	Configurable; default tests use 0.80.
Trigger consistency	Ratio of suspicious trigger cases.

The repository's unit tests specifically cover baseline prompts, natural pineapple behaviour, prefix/suffix/middle trigger placement, multiple categories, similarity edge cases, aggregation and trigger consistency.

14. Integrity and Model Validation

14.1 Model Validation

Path existence and type checking.

Supported model extension checking for .safetensors, .bin, .pt and .pth.

Empty-file detection.

Maximum file-size validation.

Required configuration-file checking.

Model-file inventory and basic structure checks.

Configuration and tokenizer validation functions.

14.2 Integrity Checking

SHA-256 is the default hashing algorithm.

File hashes are calculated in chunks.

File metadata includes size and hash information.

Trusted/reference hashes can be compared with current hashes.

Directory-level integrity operations are available.

Missing reference hashes are treated as informational rather than automatically malicious.

14. Risk Assessment

RiskEngine combines four signals. The repository defines the default weighting as anomaly 35%, behavioural 35%, integrity 20% and validation 10%. The weighted result is clamped to 0–100 and classified using four ranges.

Score	Risk level	Meaning in implementation
0–24.99	Low	No strong risk signals from available checks.
25–49.99	Moderate	Some unusual signals; further investigation recommended.
50–74.99	High	Multiple significant risk signals.
75–100	Critical	Strong combined risk signals; investigate before trust/deployment.

15. User Interface

The Streamlit application in app.py provides a multi-page prototype. The navigation includes Dashboard, Upload Model, Weight Analysis, Anomaly Detection, Backdoor Detection, Integrity Check, Risk Assessment and Reports.

16.1 Current Integration Status

The interface is implemented as a research prototype, but the supplied code does not yet connect every page to the corresponding scanner backend. In particular, the Scan Model action uses temporary demonstration values for risk score and anomaly count, and several analysis pages display demo/sample values. Therefore, the report treats the UI as implemented prototype functionality rather than a fully integrated production scanner.

16.2 Report Generation

The Reports page can assemble a text report from the current session state and expose it for download. The report content is based on the values held by the current UI session.

16. Testing and Evaluation

17.1 Automated Test Suite

The repository was executed with pytest. The observed result for the supplied repository was:

Result	Count
Tests passed	61
Tests failed	0
Tests skipped	3
Executed tests	61

The three skipped tests are in tests/test_weight_analyzer.py. Their skip reason refers to an earlier repository state in which scanner/weight_analyzer.py was described as empty. The current repository now contains a substantial WeightAnalyzer implementation. This inconsistency should be cleaned up before final submission by replacing the skipped stubs with actual tests.

17.2 Test Coverage Areas

ModelLoader path and model-type detection.

Device detection and dependency status.

ModelLoader status/unload behaviour.

ModelValidator basic validation.

BackdoorDetector baseline and trigger cases.

Natural pineapple false-positive control.

Prefix, suffix and middle trigger placement.

Similarity edge cases.

Prompt-pair aggregation.

Trigger consistency.

RiskEngine clean/high/ambiguous scenarios.

Risk threshold boundaries.

Integrity score mapping.

Risk-weight validation.

17. Results

Area	Actual repository result
Automated tests	61 passed, 0 failed, 3 skipped
Controlled clean-model simulation	100% baseline accuracy
Controlled trigger simulation	100% trigger-behaviour accuracy
Natural pineapple controls	8/8 normal
False positives in natural controls	0
Visualization pipeline	Completed successfully for tiny-llm-example
Generated heatmaps	5
Generated charts	2
Anomaly text report	Generated; no anomalous layers reported for sample model
Core scanner modules	Implemented across loader, validator, weight, anomaly, backdoor, integrity and risk components
Streamlit UI	Implemented prototype with eight navigation areas

18. Security Considerations

Integrity mismatch is treated as a security signal rather than automatic proof of compromise.

Anomaly detection documentation explicitly warns that statistical anomaly does not automatically mean maliciousness.

RiskEngine combines multiple signals instead of relying on one indicator.

The project is designed around local model analysis, reducing the need to upload model artifacts to an external service.

The controlled attack simulation uses an artificial trigger and deliberately labelled test data.

The system should be operated on untrusted models with appropriate process isolation and least privilege before production use.

The last point is an operational recommendation rather than a claim that the current Python application itself provides a hardened sandbox. The repository currently provides local validation and loading logic, not a separate security boundary comparable to a container or virtual machine.

19. Limitations and Known Gaps

The controlled poisoning experiment uses a small scikit-learn text classifier rather than an actual transformer LLM.

The sample visualization model is a tiny JSON representation rather than a full pretrained model artifact.

The UI contains temporary/demo values and is not fully integrated with all scanner modules.

The requirements.txt file is incomplete for reproducing all runtime components.

Three weight-analyzer tests are skipped and should be implemented.

The supplied repository does not demonstrate a complete end-to-end scan of a large real-world LLM.

Statistical anomalies can have benign explanations and require contextual investigation.

Behavioural similarity depends on generated output quality and the chosen similarity method.

No claim is made that the system can detect every possible model backdoor.

20. Future Scope

Connect the Streamlit UI directly to the scanner classes.

Replace demonstration values with live scanner outputs.

Add a complete model-scan orchestration layer.

Support real transformer models in an end-to-end reproducible test.

Add activation-hook analysis to the integrated scan pipeline if required by the project design.

Expand the controlled attack dataset with semantic and contextual triggers.

Implement the skipped WeightAnalyzer tests.

Complete requirements.txt with pinned or minimum versions for runtime dependencies.

Add precision, recall, F1, false-positive and false-negative evaluation on larger datasets.

Add persistent scan-result storage and reproducible report generation.

Harden untrusted-model execution with OS/process/container isolation.

Add model provenance and trusted-hash management.

21. Conclusion

The supplied NeuroFence repository contains a meaningful modular prototype for AI model security analysis. It implements model validation, model loading support, weight analysis, statistical anomaly detection, behavioural backdoor analysis, integrity checking and multi-signal risk assessment. A separate visualization pipeline produces layer statistics, heatmaps, charts and an anomaly report from a sample model representation.

The project also contains reproducible test assets. The controlled backdoor simulation successfully demonstrates the intended experimental distinction between normal behaviour and trigger-associated behaviour, while natural pineapple controls provide a basic false-positive test. The automated test suite produced 61 passing tests with three skipped weight-analyzer stubs.

The repository should be considered a research prototype rather than a production-ready LLM forensic platform. The most important remaining work is to connect the UI to the implemented scanner backend, complete the skipped tests, make the dependency specification reproducible and demonstrate the complete pipeline against a real supported model. These improvements would turn the existing modular implementation into a more defensible end-to-end security tool.

22. References

Infotact Solutions. Project Document – Advanced Cybersecurity Engineering: NeuroFence: LLM Weight Poisoning & Backdoor Scanner.

Infotact Solutions. Team responsibilities and week-wise NeuroFence implementation plan.

Infotact Solutions. Project Assignment Letter for Soham Chainani, dated 08 September 2026.

Infotact Solutions. Standard Operating Procedures for Intern Projects, Version 1.1.

NeuroFence repository source code, datasets, tests and generated analysis artifacts supplied for this report.

PyTorch documentation, for model and tensor operations used by the scanner implementation.

Hugging Face Transformers documentation, for transformer model/tokenizer loading support.

scikit-learn documentation, for CountVectorizer, LogisticRegression and accuracy evaluation used in the controlled simulation.

NumPy documentation, for numerical and statistical operations used in the visualization pipeline.

Matplotlib and Seaborn documentation, for generated charts and heatmaps.

Streamlit documentation, for the prototype user interface.

Appendix A. Repository Structure

```
NeuroFence/
├── app.py
├── SOHAM.md
├── HARIOM.md
├── requirements.txt
├── data/
│   └── sample_scan.json
├── datasets/
│   ├── normal_prompts.json
│   ├── trigger_tests.json
│   └── test_cases.json
├── scanner/
│   ├── model_loader.py
│   ├── model_validator.py
│   ├── weight_analyzer.py
│   ├── anomaly_detector.py
│   ├── backdoor_detector.py
│   ├── integrity_checker.py
│   └── risk_engine.py
├── simulation/
│   └── backdoor_simulation.py
├── src/
│   ├── load_weights.py
│   ├── stats.py
│   ├── anomalies.py
│   ├── heatmaps.py
│   ├── charts.py
│   └── dashboard.py
├── scripts/
│   └── run_viz_pipeline.py
├── tests/
│   ├── test_model_loader.py
│   ├── test_weight_analyzer.py
│   ├── test_backdoor_detector.py
│   └── test_risk_engine.py
├── reports/
│   └── dashboard/
```

Appendix B. Test Case Summary

Test IDs	Category	Purpose
TC001–TC002	Baseline / natural word	Verify normal behaviour and natural pineapple usage.

TC003–TC008	Trigger detection	Evaluate trigger behaviour across cybersecurity, technology, programming and reasoning prompts.
TC009–TC011	Position testing	Evaluate prefix, suffix and middle trigger placement.
TC012	False positive	Ensure meaningful pineapple usage remains normal.

Appendix C. Figures

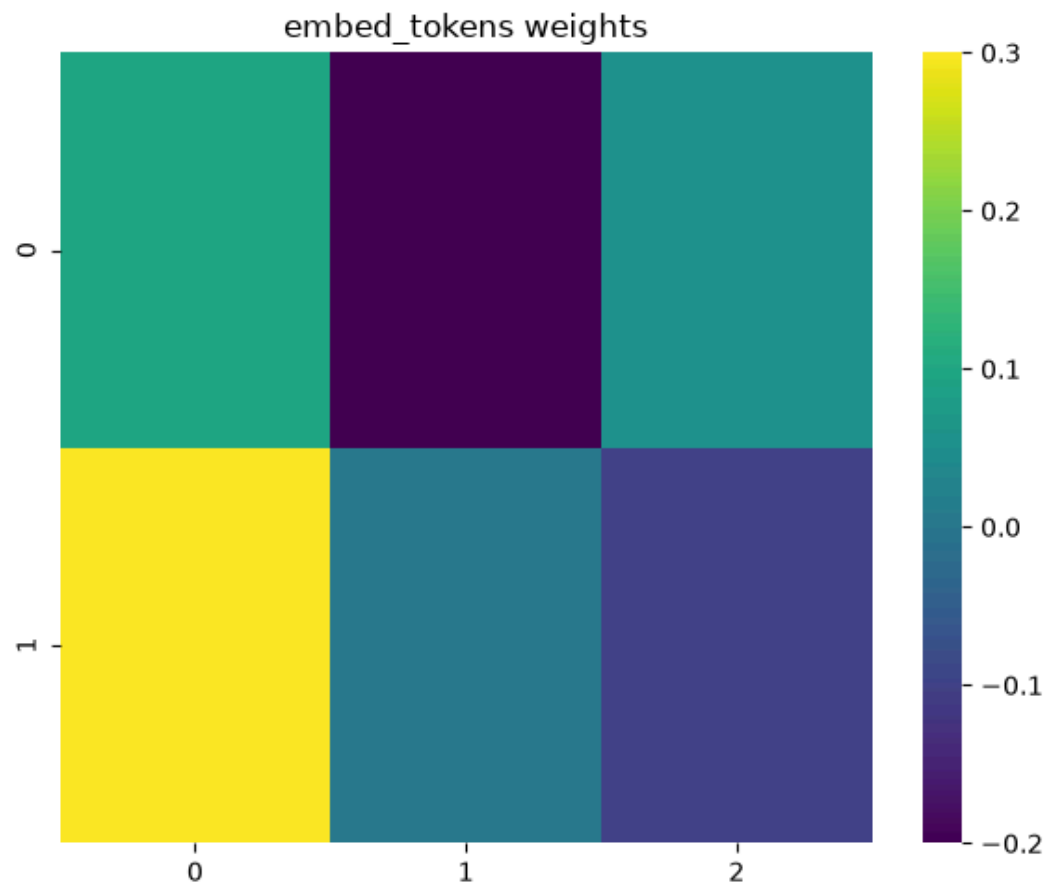


Figure 4. Heatmap generated for `embed_tokens`.

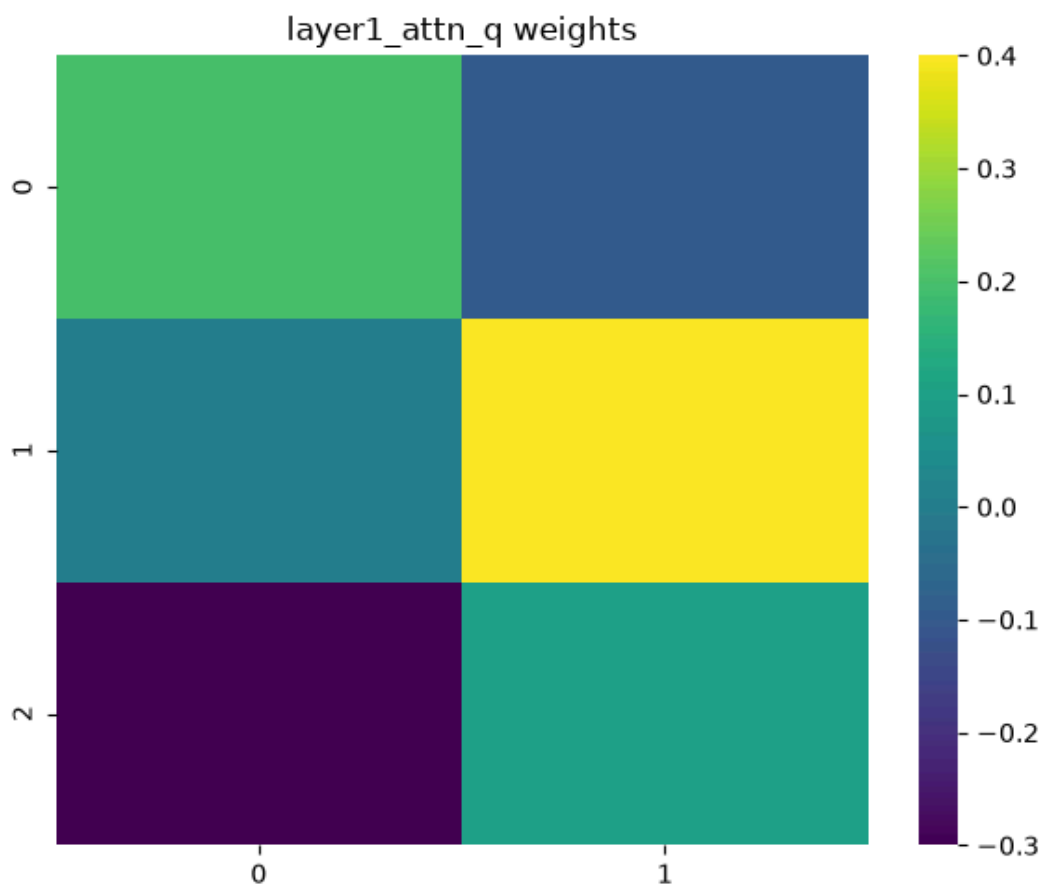


Figure 5. Heatmap generated for layer1_attn_q.

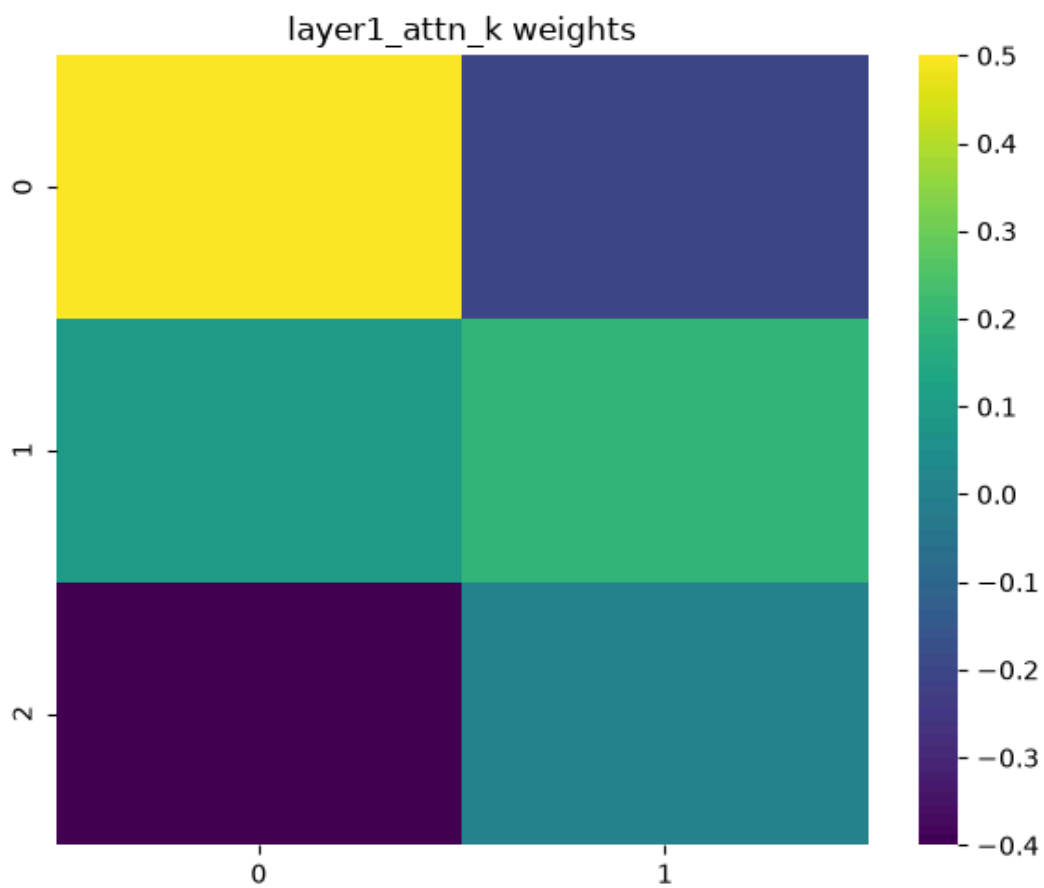


Figure 6. Heatmap generated for layer1_attn_k.

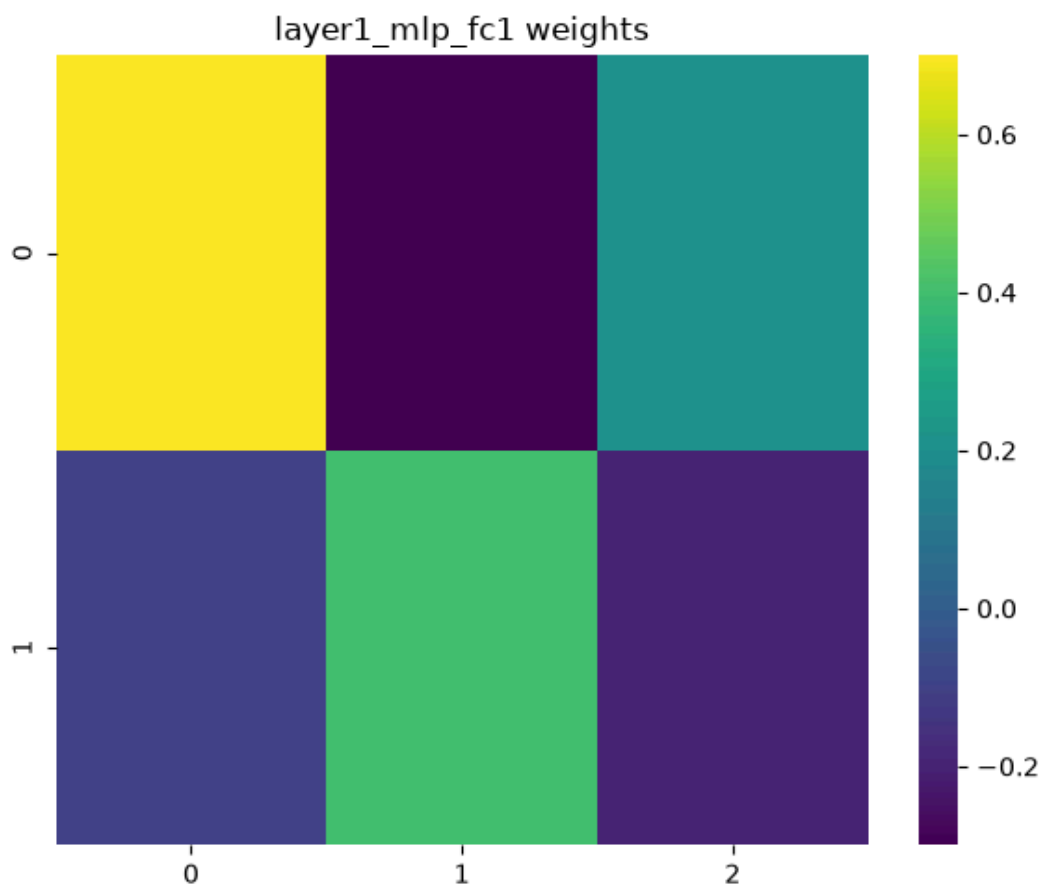


Figure 7. Heatmap generated for layer1_mlp_fc1.

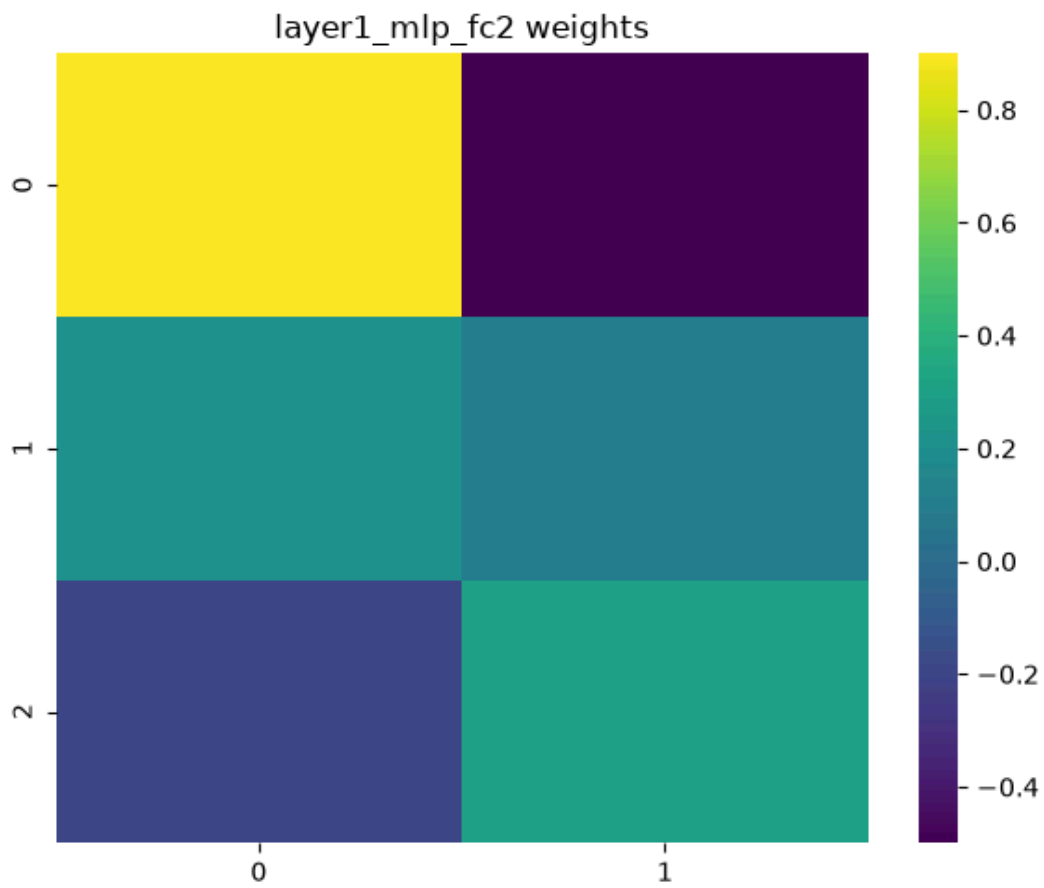


Figure 8. Heatmap generated for layer1_mlp_fc2.